Analysis of Employee Earnings from CSO Data

Muhammad Raiyan 25210283

Table of contents

1 Part 1: Analysis of CSO Earnings Dataset **2**
1.1 1. Overview 2

2 Part 2: R Package Exploration **10**
2.1 1. Overview 10

3 Part 3: Custom Function and S3 Methods – Employee Analysis **13**
3.1 1. Overview. 13

4 References **17**

1 Part 1: Analysis of CSO Earnings Dataset

1.1 1. Overview

This analysis explores the **CSO Labour Market dataset** containing information on employment, average hourly earnings, and weekly paid hours for various types of employees across different economic sectors in Ireland.

- **Categorical variables**: `economic_sector_nace_rev_2`, `type_of_employee`, `quarter`
- **Numerical variables**: `average_hourly_earnings`, `average_weekly_paid_hours`, `employment`

Dataset source:
Central Statistics Office (CSO) Ireland. "Average Earnings, Hours Worked, Employment and Labour Costs (EHQ03)". CSO Link

```
# Load required packages
library(tidyverse)  # for data manipulation and plotting
library(purrr)      # required for the project

# Load the dataset
earnings <- read_csv("earnings_cso.csv")

# View first few rows
head(earnings)
```

```
# A tibble: 6 x 6
  `Statistic Label` Quarter Economic Sector NACE Rev ~1 `Type of Employee` UNIT
  <chr>             <chr>   <chr>                       <chr>              <chr>
1 Employment        2008Q1  All NACE economic sectors   All employees      Numb~
2 Employment        2008Q1  All NACE economic sectors   Managers, profess~ Numb~
3 Employment        2008Q1  All NACE economic sectors   Clerical, sales a~ Numb~
4 Employment        2008Q1  All NACE economic sectors   Production, trans~ Numb~
5 Employment        2008Q1  Mining and quarrying (B)    All employees      Numb~
6 Employment        2008Q1  Mining and quarrying (B)    Managers, profess~ Numb~
# i abbreviated name: 1: `Economic Sector NACE Rev 2`
# i 1 more variable: VALUE <dbl>
```

1.1.1 Interpretation:

The raw CSO dataset is in a long format: each row corresponds to one statistic (for example, "Average Hourly Earnings" or "Employment") in a given quarter, sector and employee type. To make analysis easier, we will reshape it into a wide format where each statistic becomes its own column.

```
library(tidyverse)

# Pivot long dataset to wide format
earnings_wide <- earnings |>
  pivot_wider(
    id_cols = c(Quarter, `Economic Sector NACE Rev 2`, `Type of Employee`),
    names_from  = `Statistic Label`,
```

```r
    values_from = VALUE
  ) |>
  # Rename columns to simple names
  rename(
    quarter                    = Quarter,
    economic_sector_nace_rev_2 = `Economic Sector NACE Rev 2`,
    type_of_employee           = `Type of Employee`,
    average_hourly_earnings    = `Average Hourly Earnings`,
    average_weekly_paid_hours  = `Average Weekly Paid Hours`,
    employment                 = Employment
  ) |>
  # Convert categoricals to factors
  mutate(
    quarter                    = as.factor(quarter),
    economic_sector_nace_rev_2 = as.factor(economic_sector_nace_rev_2),
    type_of_employee           = as.factor(type_of_employee)
  ) |>
  # Keep only columns needed for Part 1
  select(
    quarter,
    economic_sector_nace_rev_2,
    type_of_employee,
    average_hourly_earnings,
    average_weekly_paid_hours,
    employment
  )

# Quick check
nrow(earnings_wide)
```

```
[1] 5964
```

```r
head(earnings_wide)
```

```
# A tibble: 6 x 6
  quarter economic_sector_nace_rev_2 type_of_employee     average_hourly_earni~1
  <fct>   <fct>                      <fct>                                  <dbl>
1 2008Q1  All NACE economic sectors  All employees                           21.6
2 2008Q1  All NACE economic sectors  Managers, professio~                    33.2
3 2008Q1  All NACE economic sectors  Clerical, sales and~                    16.5
4 2008Q1  All NACE economic sectors  Production, transpo~                    15.6
5 2008Q1  Mining and quarrying (B)   All employees                           21.4
6 2008Q1  Mining and quarrying (B)   Managers, professio~                    33.8
# i abbreviated name: 1: average_hourly_earnings
# i 2 more variables: average_weekly_paid_hours <dbl>, employment <dbl>
```

1.1.2 Interpretation:

- Pivoting reshapes the dataset so that each row now represents a combination of quarter, sector and employee type with all statistics stored in separate columns.

- Renaming columns makes the code shorter and easier to read.
- Converting categorical variables to factors ensures they are handled correctly in summaries and plots.

```r
library(knitr)  # for kable()

# Summary statistics of numeric variables
earnings_wide |>
  select(
    average_hourly_earnings,
    average_weekly_paid_hours,
    employment
  ) |>
  summary()
```

```
 average_hourly_earnings average_weekly_paid_hours   employment
 Min.   :11.16           Min.   :22.10             Min.   :    900
 1st Qu.:18.18           1st Qu.:30.00             1st Qu.:  28300
 Median :22.82           Median :33.80             Median :  85400
 Mean   :24.00           Mean   :33.14             Mean   : 172787
 3rd Qu.:28.57           3rd Qu.:36.58             3rd Qu.: 169000
 Max.   :54.96           Max.   :47.10             Max.   :2324400
 NA's   :4018            NA's   :4018              NA's   :4220
```

```r
# Mean hourly earnings by sector
mean_by_sector <- earnings_wide |>
  group_by(economic_sector_nace_rev_2) |>
  summarise(
    mean_hourly_earnings = mean(average_hourly_earnings, na.rm = TRUE),
    .groups = "drop"
  ) |>
  arrange(desc(mean_hourly_earnings))

# Nicely formatted table
mean_by_sector |>
  kable(caption = "Mean hourly earnings by economic sector")
```

Table 1: Mean hourly earnings by economic sector

economic_sector_nace_rev_2	mean_hourly_earnings
Education (P)	33.84837
Financial and insurance activities (K)	33.62042
Electricity, water supply and waste management (D,E)	31.22367
Financial, insurance and real estate activities (K,L)	30.53786
Information and communication (J)	30.21327
Public administration and defence; compulsory social security (O)	26.83378
Professional, scientific and technical activities (M)	25.83459
Mining and quarrying (B)	24.81287
Industry (B to E)	23.67102
Human health and social work activities (Q)	23.45102
All NACE economic sectors	23.43388

economic_sector_nace_rev_2	mean_hourly_earnings
Manufacturing (C)	23.03337
Real estate activities (L)	22.67873
Transportation and storage (H)	22.46990
Construction (F)	21.76051
Arts, entertainment and recreation (R)	18.92085
Administrative and support service activities (N)	18.80480
Wholesale and retail trade; repair of motor vehicles and motorcycles (G)	18.63878
Arts, entertainment, recreation and other service activities (R,S)	18.07449
Other service activities (S)	17.43549
Accommodation and food service activities (I)	13.72939

1.1.3 Interpretation:

- Average hourly earnings are **higher in Financial and Insurance activities**, while sectors like Accommodation and Food Services have lower wages.
- Weekly hours are fairly consistent, with minor variation across employee types.
- Employment numbers vary widely across sectors, with the highest counts in general industries and lower counts in sectors like Mining.

```r
# Remove rows with missing hourly earnings
earnings_box <- earnings_wide |>
  filter(!is.na(average_hourly_earnings))

# Boxplot of hourly earnings by employee type
p_box <- ggplot(
  data = earnings_box,
  aes(x = type_of_employee, y = average_hourly_earnings)
) +
  geom_boxplot(fill = "skyblue", width = 0.6, outlier.alpha = 0.6) +
  theme_minimal(base_size = 12) +
  labs(
    title = "Hourly Earnings by Employee Type",
    x = "Employee Type",
    y = "Average Hourly Earnings (€)"
  ) +
  theme(
    axis.text.x      = element_text(angle = 30, hjust = 1),
    axis.title.x     = element_text(margin = margin(t = 10)),
    plot.title       = element_text(face = "bold", hjust = 0.5),
    panel.grid.minor = element_blank()
  )

p_box
```

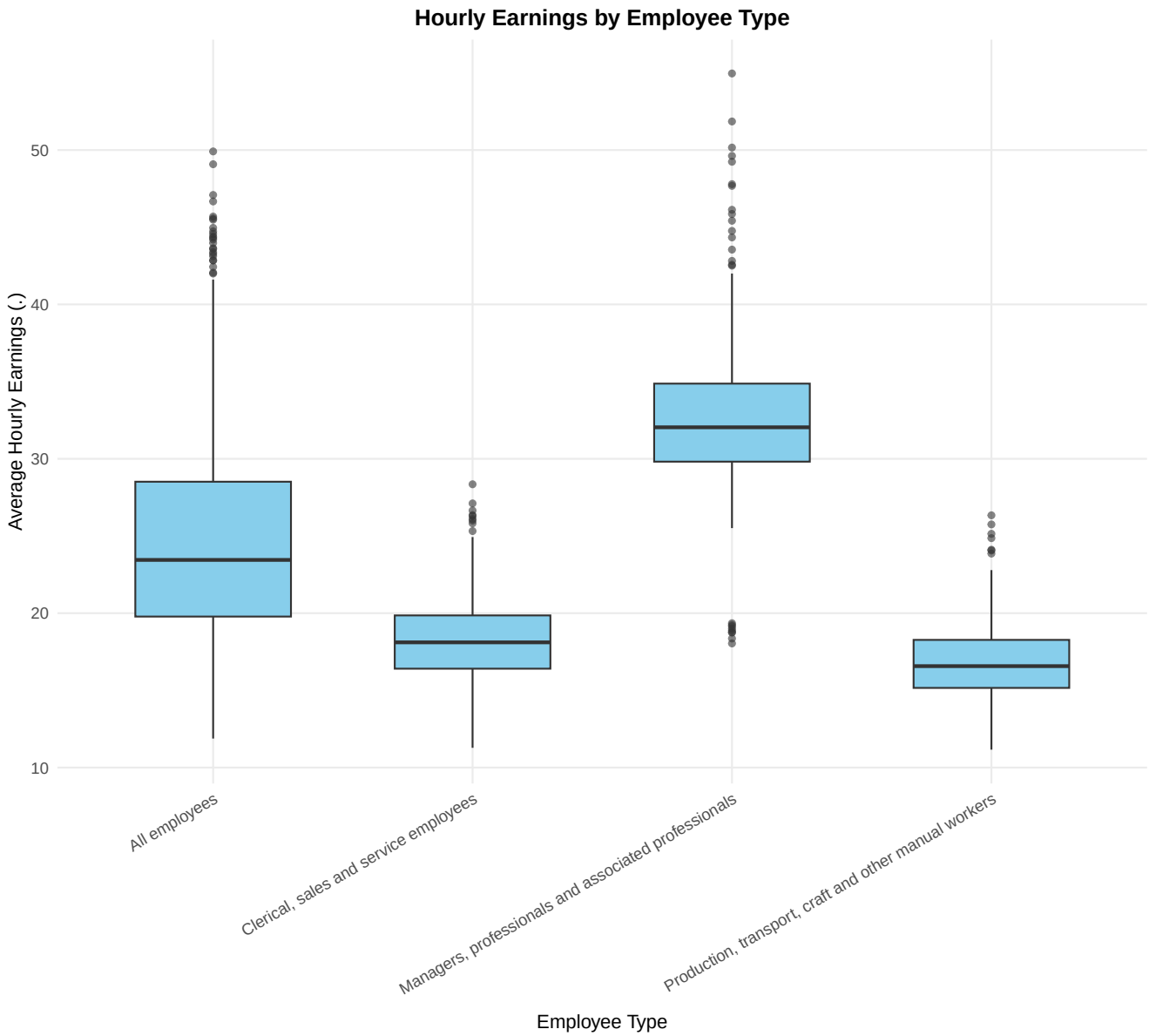

Figure 1: Hourly Earnings by Employee Type

1.1.4 Interpretation:

- Managers and professionals earn the **highest hourly wages**.
- Clerical, sales, service, and manual workers have lower wages with less variability.
- Boxplot reveals the **spread and median** of earnings across employee types.

```
# Remove any rows with missing values in the variables used
earnings_scatter <- earnings_wide |>
  filter(
    !is.na(average_weekly_paid_hours),
    !is.na(average_hourly_earnings)
  )
```

```r
# Check there are rows left
nrow(earnings_scatter)
```

```
[1] 1946
```

```r
# Scatter plot of weekly paid hours vs hourly earnings
ggplot(
  data = earnings_scatter,
  aes(
    x = average_weekly_paid_hours,
    y = average_hourly_earnings,
    color = type_of_employee
  )
) +
  geom_point(alpha = 0.7, size = 2) +
  theme_minimal(base_size = 12) +
  labs(
    title = "Hourly Earnings vs Weekly Paid Hours",
    x = "Average Weekly Paid Hours",
    y = "Average Hourly Earnings (€)",
    color = "Employee Type"
  ) +
  theme(
    plot.title       = element_text(face = "bold", hjust = 0.5),
    panel.grid.minor = element_blank()
  )
```

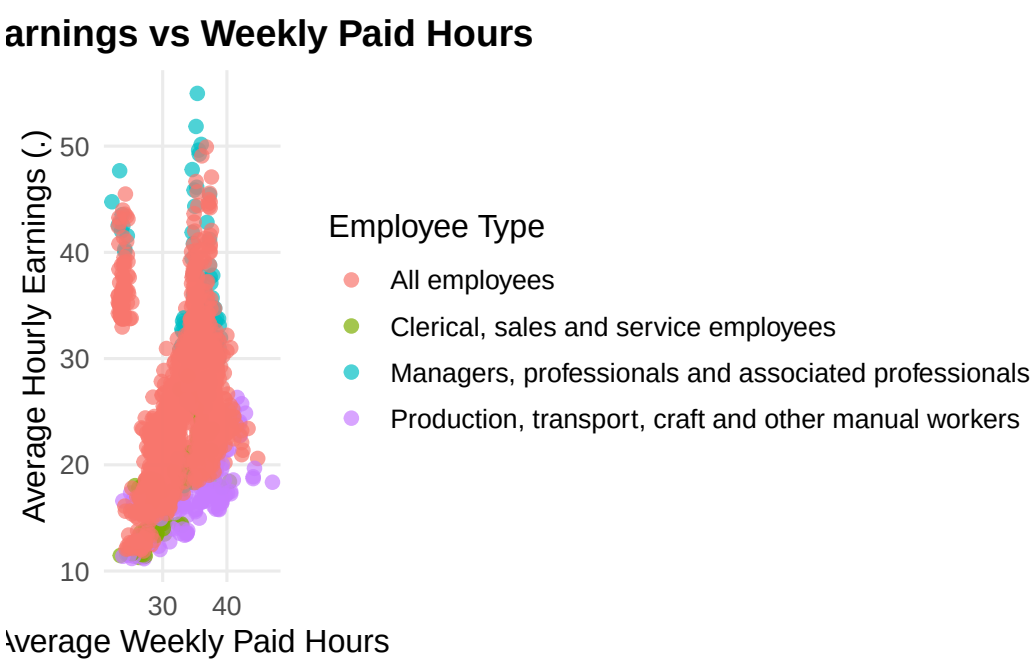

Figure 2: Relationship between weekly paid hours and hourly earnings by employee type

1.1.5 Interpretation:

- Each point represents a combination of sector, quarter and employee type, showing its weekly paid hours and hourly earnings.
- The colouring by employee type helps to see whether some groups tend to work longer hours or earn higher wages.
- The plot suggests that employee types with higher earnings generally do not have extremely low weekly hours, while some lower-earning groups may cluster at lower hourly earnings.

```r
library(purrr)
library(knitr)  # for kable()

# Numeric columns
num_cols <- c("average_hourly_earnings",
              "average_weekly_paid_hours",
              "employment")

# Calculate mean of each numeric variable by sector
summary_by_sector <- earnings_wide |>
  group_by(economic_sector_nace_rev_2) |>
  summarise(
    across(all_of(num_cols), ~ mean(.x, na.rm = TRUE)),
    .groups = "drop"
  )

# Nicely formatted table of means by sector
summary_by_sector |>
  kable(caption = "Mean earnings, hours and employment by economic sector")
```

Table 2: Mean earnings, hours and employment by economic sector

economic_sector_nace_rev_2	average_hourly_earnings	average_weekly_paid_hours	employment
Accommodation and food service activities (I)	13.72939	27.08673	136787.209
Administrative and support service activities (N)	18.80480	31.10408	78595.349
All NACE economic sectors	23.43388	32.24490	1589916.279
Arts, entertainment and recreation (R)	18.92085	28.40141	26873.239
Arts, entertainment, recreation and other service activities (R,S)	18.07449	29.07653	45080.233
Construction (F)	21.76051	36.25714	88470.930
Education (P)	33.84837	24.33776	139383.721
Electricity, water supply and waste management (D,E)	31.22367	37.55816	14290.698
Financial and insurance activities (K)	33.62042	34.94225	88338.028
Financial, insurance and real estate activities (K,L)	30.53786	34.06837	87462.791
Human health and social work activities (Q)	23.45102	30.87551	216431.395
Industry (B to E)	23.67102	37.71122	200166.279
Information and communication (J)	30.21327	36.31633	65893.023
Manufacturing (C)	23.03337	37.68265	180794.186
Mining and quarrying (B)	24.81287	39.41064	5167.857

economic_sector_nace_rev_2	average_hourly_earnings	average_weekly_paid_hours	employment
Other service activities (S)	17.43549	28.72817	24059.155
Professional, scientific and technical activities (M)	25.83459	33.16327	84458.140
Public administration and defence; compulsory social security (O)	26.83378	35.87143	115983.721
Real estate activities (L)	22.67873	31.08169	10895.775
Transportation and storage (H)	22.46990	36.33878	65106.977
Wholesale and retail trade; repair of motor vehicles and motorcycles (G)	18.63878	31.26327	266106.977

```r
# Column-wise mean across sectors using purrr
overall_means <- summary_by_sector |>
  select(all_of(num_cols)) |>
  map_dbl(mean)

overall_means
```

```
  average_hourly_earnings average_weekly_paid_hours                employment
                 23.95367                  33.02478               168107.71244
```

1.1.6 Interpretation

- The table summarises, for each sector, the average hourly earnings, weekly paid hours and employment.
- Using purrr::map_dbl(), we compute the overall mean of each numeric variable across all sectors.
- This demonstrates functional programming with purrr and provides a concise overview of typical earnings, hours and employment levels across the dataset.

1.1.7 Summary of Part 1

- The CSO earnings dataset was reshaped into a wide format and cleaned for analysis.
- Numerical summaries and tables highlighted differences in average hourly earnings across sectors.
- Boxplots and scatter plots visualised how earnings vary by employee type and weekly hours.
- A purrr-based summary provided compact overall statistics, satisfying the requirement to use purrr.

2 Part 2: R Package Exploration

2.1 1. Overview

This part explores the **lubridate** R package, which provides convenient tools for working with dates and times. The aim is to show how lubridate can simplify common tasks such as converting character strings to dates and extracting components like month and weekday.

We first introduce the package and its authors, then create a small example dataset containing date strings and numeric values. Using this dataset, we:

- convert the strings to proper `Date` objects with `ymd()`,
- extract the month for each date using `month()`,
- extract the weekday using `wday()`,
- and compute a simple summary of the values by month.

These examples demonstrate the core functionality of lubridate and how it can be integrated into everyday data analysis workflows.

```r
library(lubridate)
citation("lubridate")
```

```
To cite lubridate in publications use:

  Garrett Grolemund, Hadley Wickham (2011). Dates and Times Made Easy
  with lubridate. Journal of Statistical Software, 40(3), 1-25. URL
  https://www.jstatsoft.org/v40/i03/.

A BibTeX entry for LaTeX users is

  @Article{,
    title = {Dates and Times Made Easy with {lubridate}},
    author = {Garrett Grolemund and Hadley Wickham},
    journal = {Journal of Statistical Software},
    year = {2011},
    volume = {40},
    number = {3},
    pages = {1--25},
    url = {https://www.jstatsoft.org/v40/i03/},
  }
```

In words: the *lubridate* package (developed by Garrett Grolemund and Hadley Wickham) simplifies working with dates and times in R. It is available on **CRAN** at https://CRAN.R-project.org/package=lubridate .

We will demonstrate three key functions:

1. ymd() – convert strings to Date objects,
2. month() – extract the month from a date,
3. wday() – extract the weekday from a date.

```r
# Example dataset with dates and some values
dates_df <- data.frame(
  date_string = c("2025-01-01", "2025-02-15", "2025-03-20", "2025-04-10"),
  value = c(10, 15, 12, 20)
)
dates_df
```

```
  date_string value
1  2025-01-01    10
2  2025-02-15    15
3  2025-03-20    12
4  2025-04-10    20
```

2.1.1 Interpretation:

We create a small dataset with date strings and associated numeric values. These date strings will be converted to proper R Date objects so that we can easily extract components such as month and weekday.

```r
# 1. Convert character strings to Date objects using ymd()
dates_df <- dates_df |>
  transform(
    date_parsed = ymd(date_string)
  )

dates_df
```

```
  date_string value date_parsed
1  2025-01-01    10  2025-01-01
2  2025-02-15    15  2025-02-15
3  2025-03-20    12  2025-03-20
4  2025-04-10    20  2025-04-10
```

2.1.2 Interpretation:

- ymd() takes character strings in "year-month-day" format and converts them into R Date objects.
- The new date_parsed column is now stored in a date format that R understands, which is essential for any time-based analysis.

```r
# 2. Extract month (label = TRUE gives abbreviated names)
dates_df <- dates_df |>
  transform(
    month   = month(date_parsed, label = TRUE),
    weekday = wday(date_parsed, label = TRUE)
  )

dates_df
```

```
  date_string value date_parsed month weekday
1  2025-01-01    10  2025-01-01   Jan     Wed
2  2025-02-15    15  2025-02-15   Feb     Sat
3  2025-03-20    12  2025-03-20   Mar     Thu
```

```
4  2025-04-10    20  2025-04-10   Apr     Thu
```

2.1.3 Interpretation

- **month()** extracts the month from a **Date**. With l**abel = TRUE**, we obtain abbreviated month names (e.g. "Jan", "Feb") instead of numbers.
- **wday()** extracts the day of the week. With **label = TRUE**, we get short weekday names (e.g. "Mon", "Tue").
- These functions make it easy to summarise data by month or weekday.

```r
# Average value by month
monthly_summary <- dates_df |>
  dplyr::group_by(month) |>
  dplyr::summarise(
    mean_value = mean(value),
    .groups = "drop"
  )

monthly_summary
```

```
# A tibble: 4 x 2
  month mean_value
  <ord>      <dbl>
1 Jan           10
2 Feb           15
3 Mar           12
4 Apr           20
```

2.1.4

2.1.5 Interpretation

- Here we use the extracted month column to compute the average of value for each month.
- This illustrates how the outputs of ymd(), month() and wday() can be combined with standard R tools (such as dplyr::summarise) to perform time-based summaries.

2.1.6 Summary and Conclusion for Part 2

- ymd() converts character strings into proper Date objects.
- month() and wday() extract month and weekday information in a readable form.
- Together, these lubridate functions make it much easier to manipulate and summarise time-based data in R.

3 Part 3: Custom Function and S3 Methods – Employee Analysis

3.1 1. Overview.

This section demonstrates how to create a custom R function to analyse the CSO earnings dataset.

- We define a function employee_analysis() to summarise a numeric variable by employee type.
- We assign an S3 class and implement custom print, summary and plot methods.
- This satisfies the programming and S3 method requirements for Part 3.

```
# Custom function to summarise a numeric variable by employee type
employee_analysis <- function(data, numeric_var) {
  # Check that the chosen variable exists in the data
  if (!numeric_var %in% names(data)) {
    stop("Variable not found in dataset")
  }

  # Summary statistics by employee type
  summary_df <- data |>
    dplyr::group_by(type_of_employee) |>
    dplyr::summarise(
      mean   = mean(.data[[numeric_var]],   na.rm = TRUE),
      median = median(.data[[numeric_var]], na.rm = TRUE),
      sd     = sd(.data[[numeric_var]],     na.rm = TRUE),
      min    = min(.data[[numeric_var]],    na.rm = TRUE),
      max    = max(.data[[numeric_var]],    na.rm = TRUE),
      .groups = "drop"
    )

  # Create S3 object
  result <- list(
    variable = numeric_var,
    summary  = summary_df,
    data     = data
  )

  class(result) <- "employee_analysis"
  return(result)
}
```

3.1.1 Interpretation

The function employee_analysis() takes a dataset and the name of a numeric variable of interest. It checks that the variable exists, computes mean, median, standard deviation, minimum and maximum by type_of_employee, and returns these results in an object of class "employee_analysis".

```
# Print method for employee_analysis objects
print.employee_analysis <- function(x, ...) {
  cat("Employee Analysis - Variable:", x$variable, "\n")
```

```r
  print(x$summary)
}

# Summary method: adds extra statistics (range, coefficient of variation)
summary.employee_analysis <- function(object, ...) {
  object$summary |>
    dplyr::mutate(
      range = max - min,
      coefficient_of_variation = sd / mean
    )
}

# Plot method: horizontal bar chart of mean values by employee type
plot.employee_analysis <- function(x, ...) {
  # Make a nicer label from the variable name (remove underscores)
  var_label <- gsub("_", " ", x$variable)

  ggplot(
    data = x$summary,
    aes(x = mean, y = type_of_employee, fill = type_of_employee)
  ) +
    geom_col(width = 0.6) +
    theme_minimal(base_size = 11) +
    labs(
      title = paste("Mean", var_label, "by Employee Type"),
      x = paste("Mean", var_label),
      y = "Employee Type"
    ) +
    theme(
      legend.position = "none",
      panel.grid.minor = element_blank(),
      plot.title       = element_text(face = "bold", hjust = 0.5)
    )
}
```

3.1.2 Interpretation

- print.employee_analysis() shows a short header followed by the basic summary table.
- summary.employee_analysis() returns an enhanced table that adds the range (max – min) and the coefficient of variation (sd / mean) for each employee type.
- plot.employee_analysis() produces a bar plot of mean values by employee type, formatted to be easy to read in the PDF.

```r
# Create an employee_analysis object
result <- employee_analysis(earnings_wide, "average_hourly_earnings")

# Show the basic summary in the console
print(result)
```

```
Employee Analysis - Variable: average_hourly_earnings
# A tibble: 4 x 6
  type_of_employee                                   mean median    sd   min   max
  <fct>                                             <dbl>  <dbl> <dbl> <dbl> <dbl>
1 All employees                                      24.4   23.4  6.75  11.9  49.9
2 Clerical, sales and service employees              18.2   18.1  3.66  11.3  28.4
3 Managers, professionals and associated profess~    32.9   32.0  6.81  18.0  55.0
4 Production, transport, craft and other manual ~    16.7   16.6  3.00  11.2  26.3
```

```r
# Show the detailed summary table
summary(result)
```

```
# A tibble: 4 x 8
  type_of_employee    mean median    sd   min   max range coefficient_of_varia~1
  <fct>              <dbl>  <dbl> <dbl> <dbl> <dbl> <dbl>                  <dbl>
1 All employees       24.4   23.4  6.75  11.9  49.9  38.0                  0.276
2 Clerical, sales a~  18.2   18.1  3.66  11.3  28.4  17.1                  0.201
3 Managers, profess~  32.9   32.0  6.81  18.0  55.0  36.9                  0.207
4 Production, trans~  16.7   16.6  3.00  11.2  26.3  15.2                  0.180
# i abbreviated name: 1: coefficient_of_variation
```

```r
# This line actually produces the plot in the document
plot(result)
```

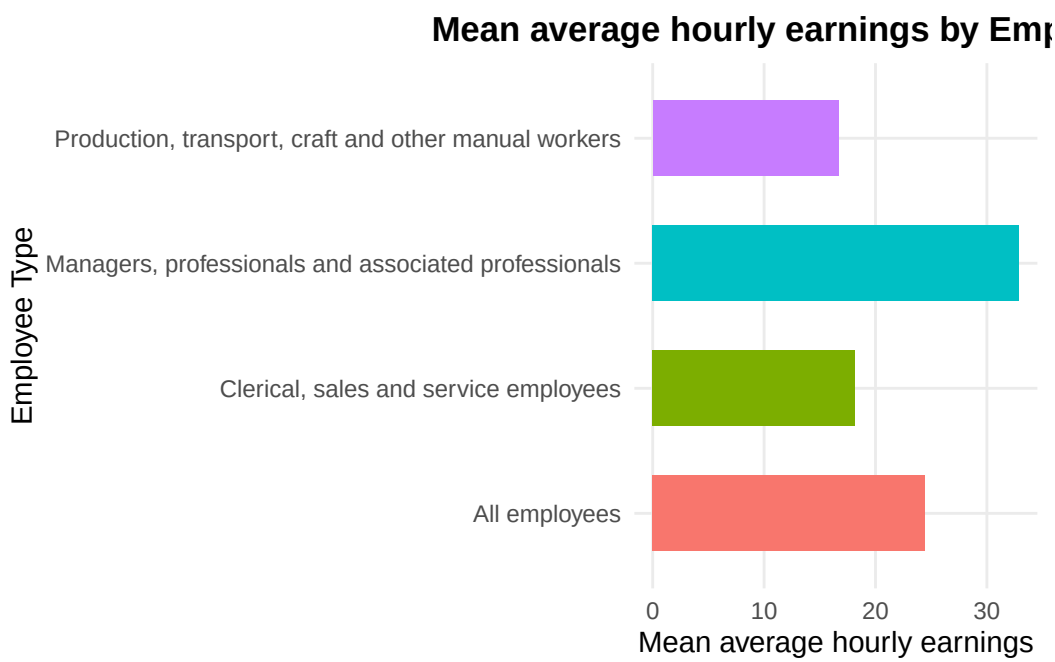

Figure 3: Mean average hourly earnings by employee type

3.1.3 Interpretation

- The employee_analysis() function is applied to average_hourly_earnings, producing an S3 object containing the summary by employee type.
- The print() method displays the basic table of means, medians, standard deviations, minima and maxima.

- The summary() method adds range and coefficient of variation, giving extra insight into variability within each employee group.
- The plot() method visualises the mean hourly earnings across employee types in a clear bar chart.

3.1.4 Summary of Part 3

- A custom S3 class “employee_analysis” was created to summarise earnings by employee type.
- Dedicated print, summary and plot methods provide different views of the results: textual summary, enhanced statistics, and a graphical display.
- The example demonstrates how S3 methods can be used to build a small, coherent analysis tool in R.

---

4 References

- Central Statistics Office (CSO) Ireland – Earnings, Hours Worked, Employment and Labour Costs (EHQ03) https://data.cso.ie/table/EHQ03
- Wickham, H. (2014). Tidy Data. Journal of Statistical Software, 59(10), 1–23. https://doi.org/10.18637/jss.v059.i10
- Wickham, H., & Grolemund, G. (2017). R for Data Science. O'Reilly Media. https://r4ds.had.co.nz
- Grolemund, G., & Wickham, H. (2011). lubridate: Make Dealing with Dates a Little Easier. CRAN R Package. https://cran.r-project.org/package=lubridate